300 ms

30% → 180 ms → Reg. X.3 → 60 ms

60% → I/O → 120 ms.

Pipeline : S.

mon pipeline : $S \times \frac{1}{8}$.

Pipeline : $\frac{1}{8}$

Temps exécution idéal : 100 cycles

: 80 + 20

↓ Calcul

↓ Accès mémoire

30 % miss cache

L2 (10).

$t_{ex} = 100 + 10 \times 10 = 200$ cycles

t_{ex} : $t_{ex\,idéal}$ + NB cache Miss × $t_{accès\,Dram}$

$A(i,j) = B(3,8) \times C(i,j)$

④ ① ③ ②

$t_{ex\,idéal}$: $4 \times 64 \times 64$.

ligne de code 64 :

B(0,0) → 2 cache miss.

B(0,1) ...

B(0,63) → 2 cache miss.

B(1,0) → cache miss

donc A, B, C → 64.

temps exécution : $\frac{1}{f}$ = $f = 1\,GHz$.

$t_{ex} = \frac{1}{f}\,(2^{14}) + (3 \times 64 \times 10) \times 10^{-9}$.

$= 16384 \times 10^{-9} + 1920\,10^{-9}$

---

write non [100] write through.

t_{ex} = 2 + 4 cache Miss × $t_{accès\,Dram}$. (1, 3, 5, 6)

write back : $t_{ex} = 6$.

Pstatique : (Pstatique registre + Pstatique add:)

le plus important.

64 (registre) × (2,8 µA) × 30 × 0,5

3 cellules → 5 Portes Log.

$32 \times 5 \times (2{,}8\ nA) + N.k_{design}$ (1) . I leak

le plus important.

Pdynamique : $P_{reg} + P_{add}$.

$64 \times P_{clock} + (16+6) \times P_{\phi}$

+.

$32 \times P_{Commutation\ add\ 1\ bit}$

S → 0101

A → 1010

1111 ← sans retenue.

A → 0.

S 0101

B → 0

→ A 1010

Porte AND : A B out

$P(A=1) = 1/2$ $P(B=1) = 1/2$

$P_{01} = P_{rb}(out=0) . P_{rb}(out=1)$

$= 3/4 . 1/4 = 3/16$

$P_{comb} = P_{rob\ 0\to1} \times P_{grille}$.

$= 3/16 \times P_{grille}$.

Arch 1 : P_{dyn} : $64 \times P_{clock} + \frac{1}{4}$ $P_{\phi} \times 64$ (basculement)

$I=0$ $\frac{1}{2}$ $I=1$ $\frac{1}{2}$

P_{dyn} : $32 \times P_{comp\ 1\ bits}$.

Archi 2 : (32 bits)

Sram.

bit rés : A_{3i} B_{3i}
0 0
0 1 | comparaison cyclo. 1) $A = B$ 2) $A < B$
1 0
1 1 comparaison cyclo

$P_{reg\ dyn} : \frac{1}{2}(2\ P_{clock} + \frac{1}{4} P_R \times 2)$ 2 boucles.
$+ \frac{1}{4}(64\ P_{clock} + \frac{64}{4} P_R)$.

Banque div/2. dessin cycle

$P_{comparateur} : \frac{1}{2}(P_{comp\ 1\ bit}) + \frac{1}{2}(32 \times P_{cyc})$ complète

---

Dram :

cycle 1 : I_0, I_1
cycle 2 : I_2,
cycle 3 : I_3,
cycle 4 : I_4.
cycle 5 : I_5.
cycle 6 : I_6
⋮

Destination

Etiq1 :
move r7,r3 0
load r8,(r3) 1
add r3, r3,4 2
load r9,(r3) 3
ble r8,r9,Etiq2 4

move r3,r7 5
store (r9),r3 6
add r3,r3,4 7
store (r8),r3 8
add r5,r5,1 9
Etiq2 :
add, r6,r6,1 10
add r7,r7,4 11
blt r6,r4,Etiq1 12

2 accès mem +
2 unité de calcule.

$$O_i(x,y) = \begin{cases} 1 & \text{if } |\overbrace{I_i(x,y)}^{Sn} - \overbrace{B_i(x,y)}^{I_2}| \geq \tau \\ 0 & \text{if } |I_i(x,y) - B_i(x,y)| < \tau \end{cases}$$

I_4

Puis mise à jour de l'image de fond :
$B_{i+1}(x,y) = (1-a) * B_i(x,y) + a * 1/3 (I_{i-2}(x,y) + I_{i-1}(x,y) + I_i(x,y))$

cycle 1 : load $R_1, I_1(x,y)$,
load $R_2, B_1(x,y)$,
sub R_2, R_1, R_2) a
valeur abs. R_2.
test. $a > T$ sinon
échange H.
calcul 0

1 : return 1.

cycle 1 : I_1
cycle 2 : I_2
cycle 3 : $I_3 - I_2$
cycle 4 : $|I_1 - I_2|$
cycle 5 : test.

2/2. mem uc

cycle	mem 1	mem 2	uc1	uc2
1	I_1	I_2	—	—
2	I_3	I_{10}	I_3	I_{11}
3			I_4	I_{12}
4			I_5	I_{13}
5	I_6		I_{14}	
6			I_{15}	
7	I_{16}			

Sans pipeline : 16 cycles.

Sans Cycle : 16 c
// à 4 : 7 c
// à 6 : 6 c.

cycle	mem 1	mem 2	m3	uc1	uc2	uc2
1	I_1	I_2	I_3	—	—	
2	I_{10}			I_3	I_{11}	I_{12}
3				I_4	I_{13}	
4				I_5	I_{14}	
5	I_6			I_{15}		
6	I_{16}					
7						

```
Etiq :  move R3, R7
        load R8,(R3)
        add R3, R1, 4
        load R9, (R3)
        add  R7, R1, R2
        Sub  R6, R1, 4
        ble R8,R9, Etiq
        ...
```

Exercice

Considérons un filtre de traitement d'image :

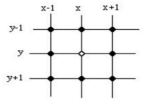
Pour tous les pixels d'une image I, cette opération consiste à remplacer la couleur de chaque pixel (x,y) par une combinaison linéaire des couleurs de tous les points immédiatement voisins (y compris lui-même) :

g(x,y)=ΣiΣj h(i,j)*I(x+i, y+j) avec i et j variant de -n à +n

$$\left(\sum_{i=-n}^{i=n}\sum_{j=-n}^{j=n} h(i,j).I(x+i,y+j)\right)$$

où h est un masque qui peut adoucir ou durcir l'image.

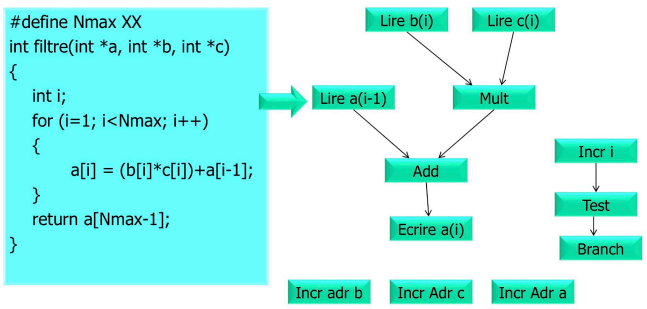
A titre d'exemple, si n=1, les pixels considérés sont positionnées comme ci-après avec un exemple de masque h.



h=1/10* | 1 1 1 | 1 2 1 | 1 1 1 |

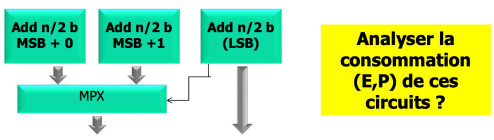
Sur les architectures précédentes :

- Relever le parallélisme intracore, puis estimer le temps de réponse pour n=1



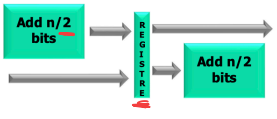
Optimisation architecturale : parallélisme, pipeline ...

- Parallélisme :



Analyser la consommation (E,P) de ces circuits ?

- Pipeline :



Consommation logicielle : Instruction Level Power Analysis

Calculer l'énergie consommée par un filtre FIR :
Y[n] = b0.X[n] + b1.X[n-1] + b2.X[n-2] + b3.X[n-3].

Dans le cas idéal (pas de pénalité)

Dans le cas pire (cache miss + rupture de pipeline, 5 étages de pipeline).

Hypothèse : les instructions s'exécutent en un cycle d'horloge.

On prendra F= 1Ghz.

Deux cas de figure :
- I variable par instruction : Iadd, Imult, Iload/store
- I constant, Pcore = 500 mW pour F = 1Ghz

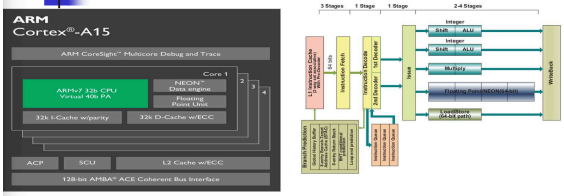
lire X_m, X_{m-1}, X_{m-2}, X_{m-3}

mult $b_0 X_m \to R_0$; $b_1 X_{m-1} \to R_1$; $b_2 X_{m-2} \to R_2$; $b_3 X_{m-3} \to R_3$

add $R_1 R_0 \to R_4$; $R_2 R_3 \to R_5$; $R_4 R_5$

Energie : $t_{cycle} (V_{dd}) (4 I_{Idle\,cu} + 4 \times I_{mult} + 3 I_{add})$

Estimation logicielle : FLPA



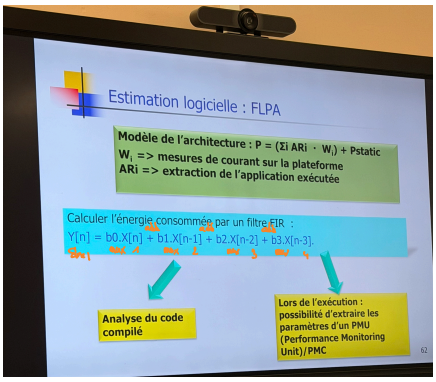
$I = I_0 + \alpha I_{4,44} + n I_{mult} + P.A. + l I_{mem}$ (α : 0, 1, 2 ; n : 0, 1 ; P.A. : 0, 1 ; l : 0, 1, 2.)

P, E = ?

1 - Identifier les paramètres architecturaux importants intervenant sur la consommation
2 - Modèle ?

Consommation → type :

float, add → mémoire :



cycl	mem 1	mem 2	mux	ALU	ACU
1	l_1	l_2			
2	l_3	l_4	R_1		
3			R_2		
4			R_3	add 1	
5			R_4		
6				add 2	
7				add 3	
8	store				

ss $I_1 = I_0 + 2 \times I_{acc\,mem}$

ss $I_2 = I_0 + 2 \times I_{acc\,mem} + I_{mem}$

ss $I_2 = I_0 + I_{mul}$

ss $I_3 = I_0 + I_{mul} + I_{add}$

ss $I_4 = I_0 + I_{mul}$

ss $I_5 = I_0 + I_{mul}$

ss $I_6 = I_0 + I_{mul}$

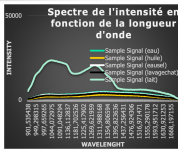
ss $I_7 = I_0 + I_{acc\,mem}$

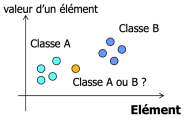
$E = I_{add} + \frac{1}{8} \times (I_1 \dots I_8)$

$E = 1^V \cdot 6^{-3} \times (\dots) = 0{,}4\,mJ$

Exemple : classification supervisée - KNN

Classification supervisée basée sur la recherche des plus proches voisins (KNN).
Exemple : détection des émotions, fruits et légumes/ sain-non sain, adultération/produit de soins (Huile essentielle).
Deux phases :
- Entrainement : création d'un dataset représentant les classes
- Classification (Inférence)





Pour l'élément D , calcul de la distance euclidienne (ou de Manhattan) par rapport à chaque classe :
$DE(C_i) = \Sigma_k |D - E_k(C_i)|$

D appartiendra à la classe ayant la plus faible DE

Exemple d' Influence du parallélisme sur la consommation

Le traitement repose sur deux boucles :
Loop1 : Acquisition d'une nouvelle donnée D, pour chaque classe Ci représentée par k éléments E_k (base d'apprentissage), calcul de la distance euclidienne :

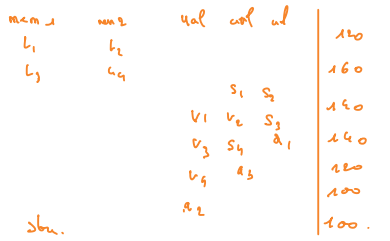
$DE(C_i) = \Sigma_k |D - E_k(C_i)|$

Si k= 4 : $DE(C_i) = |D - E_1(C_i)| + |D - E_2(C_i)| + |D - E_3(C_i)| + |D - E_4(C_i)|$

Loop 2 : L'élément D appartiendra à la classe ayant la plus faible $DE(C_i)$
Pour toutes les classes Ci :
Si $(DE(C_i) < DE)$ alors $C_{final} = C_i$; VD=VD(C_i) ;

=> Puissance et Energie Consommée ?

Que devient cette consommation en fonction :
Du nombre de classe,



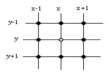
$\epsilon = 6^{-5} \times 6^{-3} (\ldots\ldots) = 700\ pJ.$

Exercice – Filtrage vidéo

Considérons un algorithme de filtrage vidéo => pour tous les pixels d'une image I, remplacer la couleur de chaque pixel (x,y) par une combinaison linéaire des couleurs de tous les points immédiatement voisins (y compris lui-même) :

$$g(x,y) = \sum_{i=-n}^{i=n} \sum_{j=-n}^{j=n} h(i,j).I(x+i, y+j)$$

Exemple n=1



où h est un masque qui peut adoucir ou durcir l'image.
Taille de l'image : 640x420. Pixels codés sur 8 bits (1 octets).

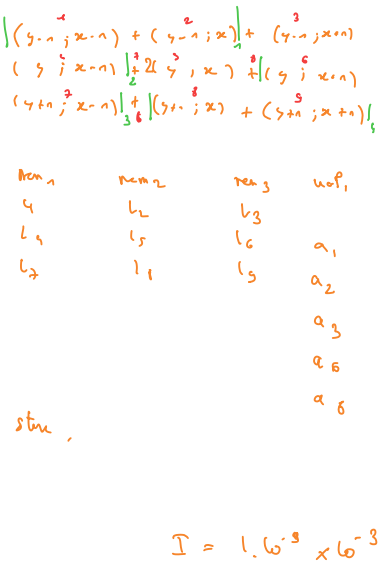
Processeur (1 core) doté d'un cache L1 dont la ligne de cache est de 32.

4.1 – Ecrire un algorithme succinct pour réaliser cette opération de filtrage.

4.2 - Déterminer le temps d'exécution de ce programme puis l'énergie consommée dans le cas le plus favorable pour n=1. (Hypothèse : CPI=1 cycle).

4.4 – Reprendre cet exercice avec un core capable de faire par cycle : 3 accès mémoire, 2 mult, 2 adds, div. (autre possibilité : Instruction MAC possible)

Instr	1	2	3	4	5	6	7	8
$P_{(mW)}$	100	120	140	160	180	200	210	220



hypothèse :
$I(x,y) \times 2 = I(x) + I(x,y)$

$I = 1.6^{-9} \times 6^{-3} (\ldots\ldots) = 0,99\ nJ.$

→ économie : - Jeu d'instruction.
- utilisation : LLPA.
FLPA.

t_1 – 2ms
t_2 – 4 ms.
t_3 – 4 ms
t_4 – 1 ms
T_5 – 1 ms.

$\frac{2.4.4.1.1.}{30} = \frac{12}{30} < 1.$

oui.
1 cœur de Pacs.

n = dim par tous exemples.

2^8

Cours : multi core :

→ loi Amdahl : $acc(n) = T_s + T_p / (T_s + T_p/n)$.

$T = T_s + T_p/n$.

boucle 1-10 :

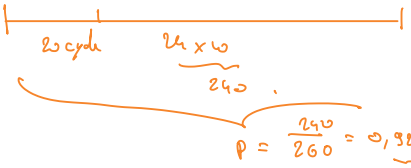
core 1	core 2	core 3	core 4
si 2 cycle :			
i = 1 → 3.	i = 4 → 7	i = 8 → 10	
tboucle × 3	tboucle × 3	tboucle × 3.	

Simd :

une instruction et synchronisé.

accélération théorique :

loi de Gustafson : $A = (1-p) + s \times p$



$p = \frac{240}{260} = 0,923$

par le //.

SIMD :

core 1	2	3	4.	
a	b	c	d	← doit attendre que H et fini pour passer.

MIMD → plus de synchronisation → tâche

ordonnancement → quand.

affectation → attribution tache affectation.

Simd → par cœur.

↘ Simd.

core				
1	T_1	T_2	T_6	T_7
2		T_3		
3		T_4		
4		T_5		

MIMD → OS qui gère ça → on fait des tâches // !

limite 4 → donc aucun intérêt pour 8/16 cœurs

Simd → utilisation qui dépend le //.

Si on écrit gauche et il faut

calculer → celui des possibilité du

cœur !

exercice filtrage :

$$g(x,y) = \sum_{i=1}^{3} \sum_{j=1}^{3} h(i,j) I(x+i, y+i)$$

avec $h = \frac{1}{16} \begin{vmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{vmatrix}$

$\begin{vmatrix} (x-1, y-1) & (x, y-1) & (x+1, y-1) \\ (x-1, y) & (x, y) & (x+1, y) \\ (x-1, y+1) & (x, y+1) & (x+1, y+1) \end{vmatrix}$ donc $3 \times 3 = 9$.

Pour $n = 1$: $(2n+1)^2 = 3^2 = 9$ pixels car

Si 9 pixels : $I_1 = I(x,y)$ .... $I_9 = I(x,y)$

$$g(x,y) = \frac{1}{16} (\underbrace{I_1 + I_2 + I_3 + I_4 + 2I_5 + I_6 + I_7 + I_8 + I_9}_{\text{16 coefficients mosaïque. 9 ⊕ contribution :}})$$

Par décomposition :

$S_1 = P_1 + P_2$
$S_2 = P_3 + P_4$
$S_3 = P_5 + P_6$
$S_4 = P_7 + P_8$

$S_5 = S_1 + S_2$
$S_6 = S_3 + S_4$
$S_7 = S_5 + S_6$
$S_8 = S_7 + P_9$ → 8 additions et 9 postes.

→ le parallélisme maximal des postes => 9. pour le parallélisme intra-core.

Estimer le temps :

Sans PiPeLine :

9 multiplications et 8 additions = 17 cycles :

$t_{sans\ PiPeLine} = T$ (17 cycles). avec $f = \frac{1}{T}$ et $f = 1\,GHz$

$t_{sans\ PiPeLine} = 17/F = 17\,ns$.

avec PiPeLine :

cycle	multiplication	addition
1	P_1	-
2	P_2	-
3	P_3	$P_1 + P_2$
4	P_4	$S + P_3$
5	P_5	$S + P_4$
6	P_6	$S + P_5$
7	P_7	$S + P_6$
8	P_8	$S + P_7$
9	P_9	$S + P_8$
10	-	$S + P_9$.

on a donc $t_{pipeline} = T$ (10 cycles) :

$t_{pipeline} = 10/F = 10\,ns$

estimation : $t_{pipeline} = 1{,}7 \times$ plus rapide que $t_{sans\ pipeline}$.

Pour n = 2 :

$N = (2n+1)^2 = 5^2 = 25$.

donc 25 lectures, 25 multiplications, 25 additions, 1 écriture.

Cas pour architecture 2 UE, 2 mem : pour n = 1.

cycle :	mem1	mem2	calcul 1	calcul 2
1	L_1	L_2	–	–
2	L_3	L_4	Π_1	Π_2
3	L_5	L_6	Π_3	Π_4
4	L_7	L_8	Π_5	Π_6
5	L_9	–	Π_7	Π_8
6	–	–	Π_9	A_1
7	–	–	A_2	A_3
8	–	–	A_4	A_5
9	–	–	A_6	A_7
10	–	–	A_8	–
11	–	–	–	–

L : lecture Π : multiplication A : addition É : écriture.

$t_{n=1,2} = 11$ Tcycle

Cas pour architecture 3 UE, 3 mémoire : pour n = 2.

cycle :	mem1	mem2	mem3	calcul 1	calcul 2	calcul 3
1	L_1	L_2	L_3	–	–	–
2	L_4	L_5	L_6	Π_1	Π_2	Π_3
3	L_7	L_8	L_9	Π_4	Π_5	Π_6
4	–	–	–	Π_7	Π_8	Π_9
5	–	–	–	A_1	A_2	A_3
6	–	–	–	A_4	A_5	A_6
7	–	–	–	A_7	–	–
8	–	–	–	A_8	–	–
9	–	–	–	–	–	–
10	–	–	–	–	–	–

L : lecture Π : multiplication A : addition É : écriture.

$t_{n=1,3} = 9$ Tcycle.

Si n = 2 donc on a 25 pixels :

Il y a 25 lectures, 25 multiplications, 24 additions, 1 écriture

Cas pour 2 UE, 2 mem :

$t_{n=2,2} = 27$ tcycle.

Cas pour 3 UE, 3 mem :

$t_{n=4,3} = 20$ tcycle.

On peut voir que si n augmente alors on a plus de parallélisme et donc le temps diminue.

Si mem est petite on est obligé d'attendre par la suite.

→ Page imprimer recto-verso.

Les calculs se font le même coup.

Exemple machine VLIW/programme

Il faut le read after write problème.

Exemple de programme :
A = J + K; B = L × M; C = N – B; D = A / B

Dépendance de type RAW (*Read After Write*) sur A et B

	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5
Accès MEM	Load Rj,J	Load Rl,L	Store Ra,A	Store Rb,B	Store Rc,C
Accès MEM	Load Rk,K	Load Rm,M	Load Rn,N		Store Rd,D
Add sub		Ra=Rj+Rk		Rc=Rn-Rb	
Mul			Rb=Rl*Rm		
Div				Rd=Ra/Rb	

Amélioration ?

Achitectures multiflots

- Déjà étudié : recherche du parallélisme intra task
- Amélioration : exécution de différents flots, parallélisme inter task

Exemple de programme :
Pgm 1 : A = J + K; B = L × M;
Pgm 2 : C = N – E; D = F/G

Exécution ?

	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle …
Accès MEM	J	L	N	F	Sto A
Accès MEM	K	M	E	G	Sto B
Add sub		J+K		N - E	
Mul			L x M		
Div					F/G

Reprendre l'application de filtrage vidéo et proposer une amélioration.

Correction : page 58.